

IMPERIAL

A Multi-Agent LLM Framework for Maintaining Code Quality in Iterative Software Development

Bernard Moy
10/09/2026

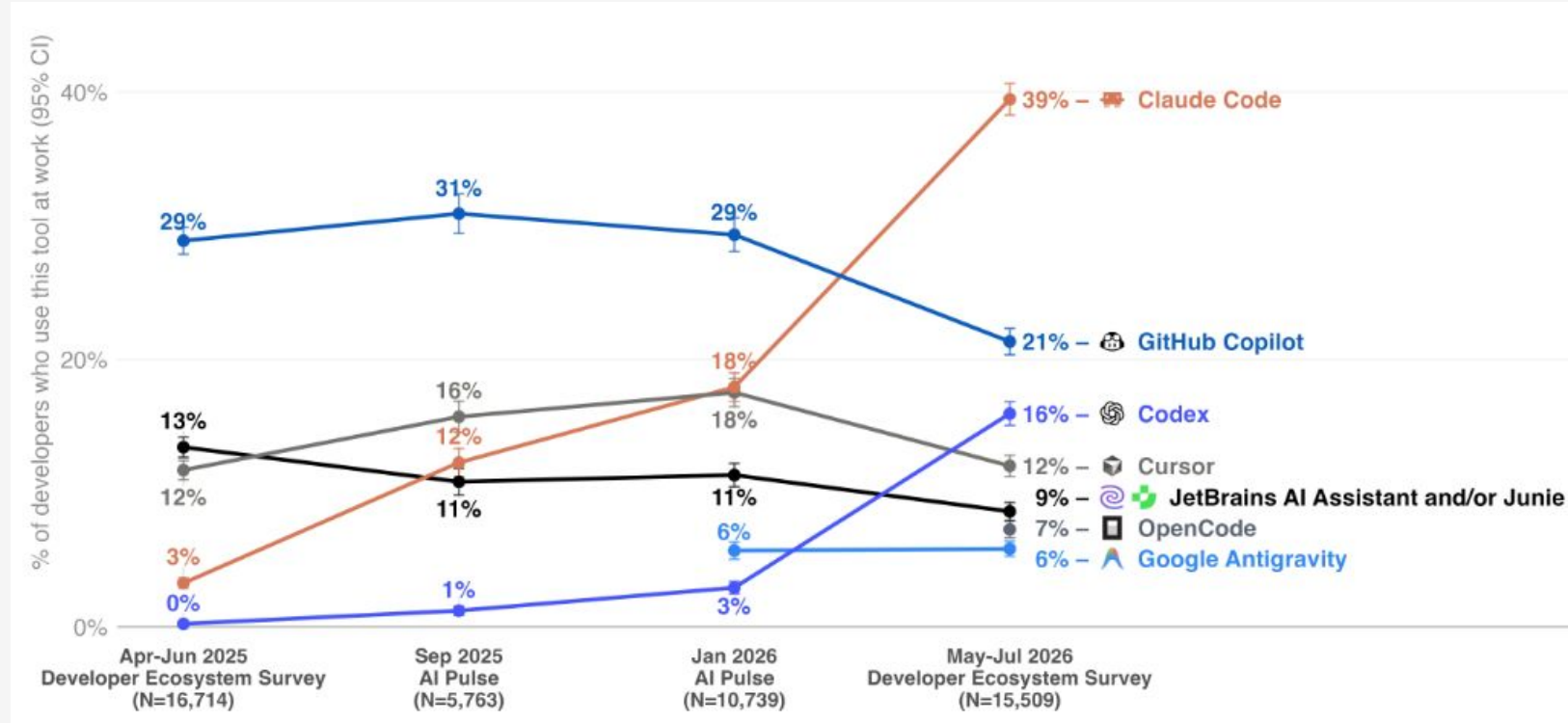
Agenda

- 1 Motivation and Objectives
- 2 Proposed Technical Architecture
- 3 Experimental Methodology
- 4 RQ1: Mitigating Technical Debt
- 5 RQ2: Correctness Trade-offs
- 6 RQ3: Influence of Human-in-the-loop
- 7 Ablation Study
- 8 Conclusions and Future Work

Motivation

Rapid Adoption of AI Coding Agents

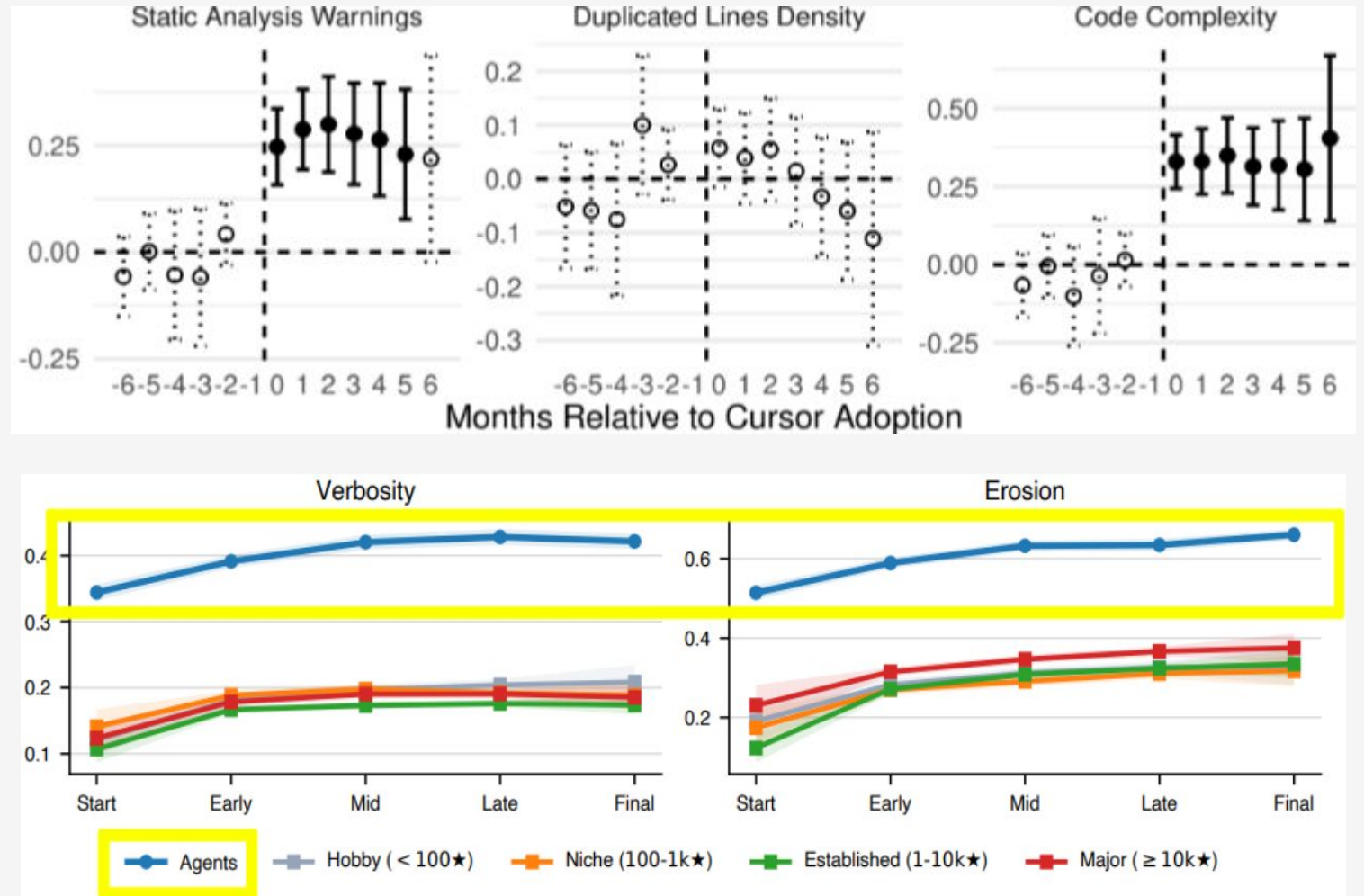
- **AI Coding Agents** are capable of reading and editing files, and can autonomously execute commands.
- **Examples:** Anthropic Claude Code, OpenAI Codex.
- **90%** of professional developers are using them weekly, with **68%** using daily¹.



Motivation

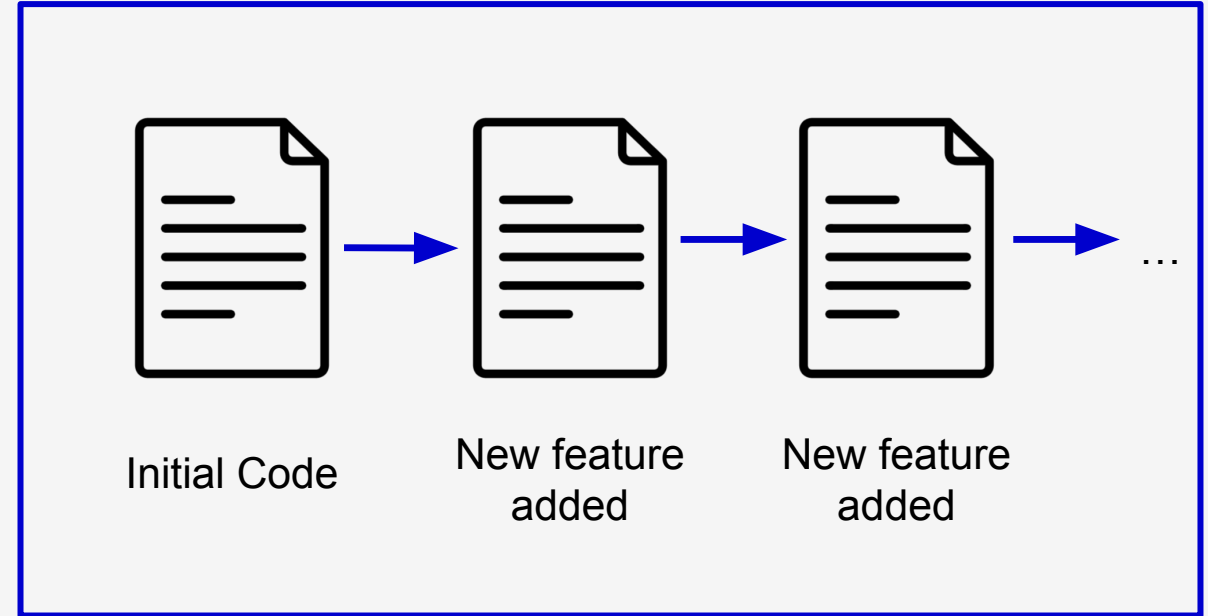
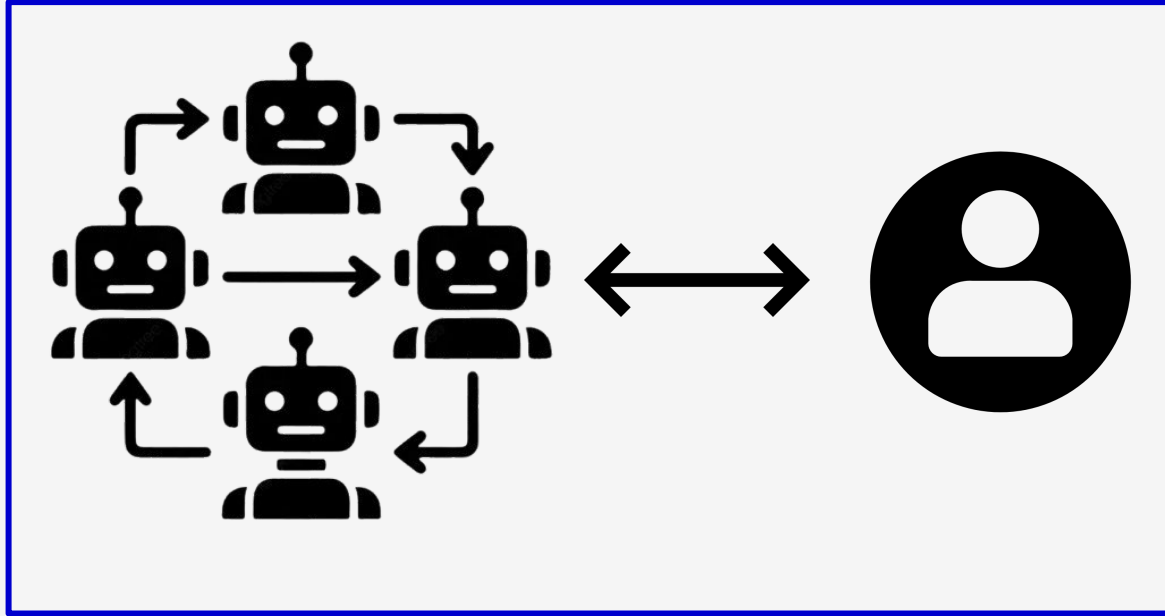
The Transient Velocity Gain

- The gain in development velocity concentrated in the **first 1-2 months²**.
- **Technical debt** catches up:
+30% static analysis warnings
+41% code complexity growth
- Agents consistently produce **lower quality code** and **degrade it faster** than human developers³.



Objectives

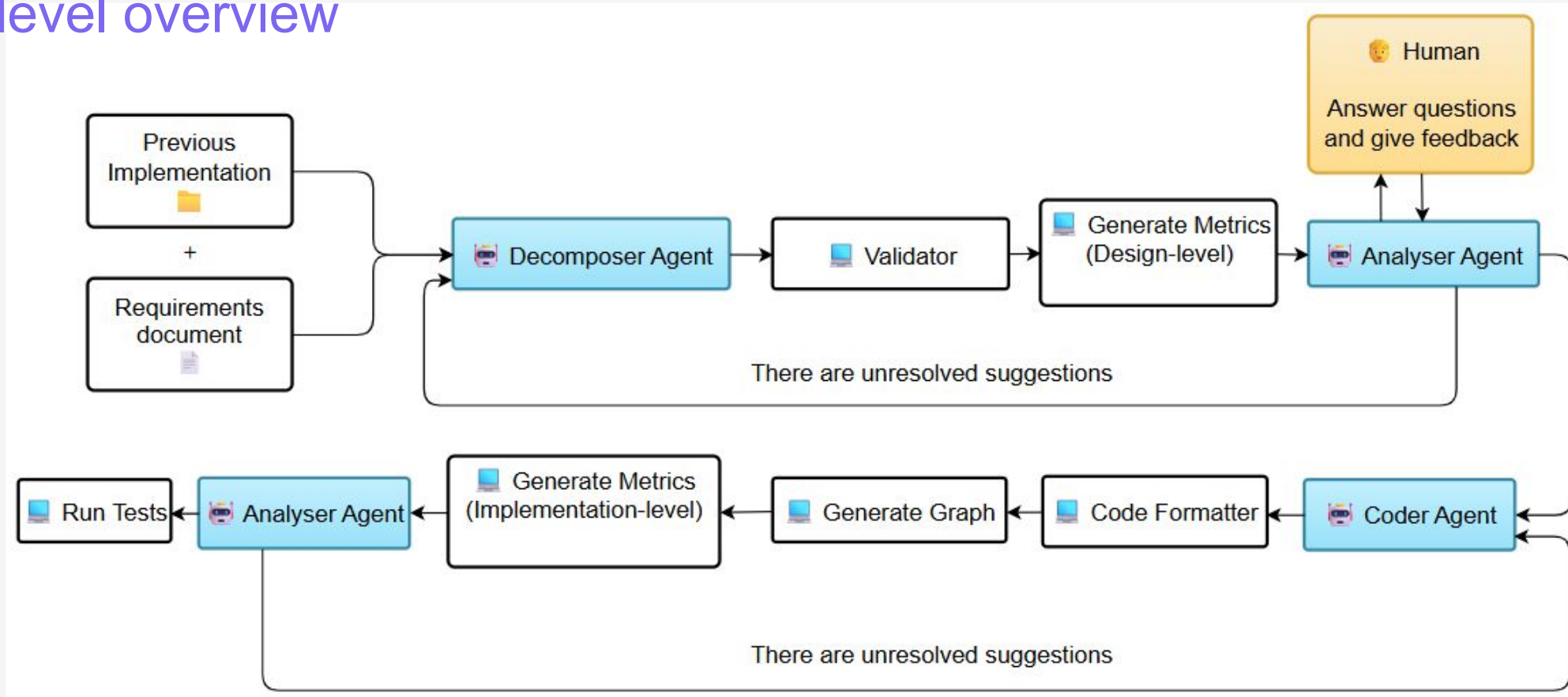
The Aims of Our Multi-Agent Framework



- Design and evaluate a **multi-agent system** with human-in-the-loop providing feedback, implemented in Python.
 - Identify suitable **benchmarks** to evaluate code quality and correctness.
-
- Mitigate the **accumulation of technical debt and code quality issues** in **iterative** development.
 - Reduce the **quality gap** between human-written and agent-written code.

Proposed Technical Architecture

High-level overview



- Feedback loops between the **decomposer** and **analyser**, and between **coder** and **analyser**, using shared structured JSON.
- The human is involved in providing additional context to justify a refactoring, and approving the proposed plan / suggestions.

Proposed Technical Architecture

Roles of Specialised LLM Agents

Decomposer

- Reads requirements and previous code, **proposes modular design** using the principle of information hiding with modules reuse.
- Outputs high-level (dependency graph), low-level (public interfaces).

Analyser

- Critically evaluates the proposed design and computed metrics.
- Discovers maintainability issues and **provide suggestions**.

Coder

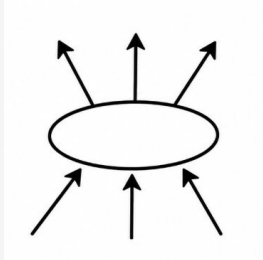
- **Implements selected modules** according to the design.
- The dependency graph and implementation-level metrics are then re-generated.

Proposed Technical Architecture

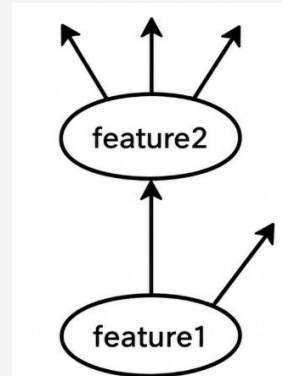
Code Quality Metrics as Guidance

- Static analysis metrics problems: False positives, limited architectural quality insights⁴.
- We therefore leverage LLM's reasoning ability with few-shot examples, with metrics as **guidances**.

Design-level metrics detectable early



Hub-like
modules



Unstable
dependencies

Implementation-level metrics

```
def parse_input(raw, w, mode="2val"):
    if mode == "3val":
        compact = raw.replace(" ", "")
        if w == 1 and compact.lower() == "x":
            return "X", 1
        if w == 1 and compact in ("0", "1"):
            return compact, 1
        m = re.fullmatch(r"(?i)(\d+)('[bhd])([0-9a-fx_]+)", compact)
        if m:
            bits = int(m.group(1))
            base = m.group(2).lower()
```

Cyclomatic complexity, lines of code,
lack of cohesion of methods (LCOM)

Proposed Technical Architecture

Role of Human-in-the-loop

```
def write_anomalies_csv(anomalies: list[SegmentationAnomaly], output_path: str) -> None:
    def write_anomalies_csv(anomalies: list[SegmentationAnomaly], output_path: str) -> None:
        with open(output_path, "w", newline="", encoding="utf-8") as output:
            writer = csv.DictWriter(output, fieldnames=ANOMALY_FIELDS)
            writer.writeheader()
            writer.writerows({f: getattr(a, f) for f in ANOMALY_FIELDS} for a in anomalies)
```

Q: Extract a shared helper module to resolve duplicated code?

Consider:

- Is the duplication a core subdomain?
- Will the duplications diverge?
- Does it justify creating a new module (trade-offs)?

- Resolving software quality issues often involve **additional context information** or **trade-offs**.
- The termination condition of the feedback loops also **lacks universal thresholds**, unlike prior work using test or compilation rates as the condition.
- Therefore, human is involved in **providing context** and **approving plans**.

Experimental Setup

Benchmarks: SlopCodeBench + Popular GitHub Repositories

- **SlopCodeBench** features 36 CLI problems across 196 checkpoints.
 - Allows assessment of how code quality evolves over time.
 - Uses **black-box testing** for correctness evaluation.
- **8 GitHub Python repositories** for comparison to human-written code, from 8k to 100k+ stars (e.g. Flask, FastAPI).
 - Analysed their **source files**.
 - Extracted **historical version snapshots** to assess how code quality evolves over time.

Problems are implemented in Python, using the GPT-5.6-luna and GPT-5.4-mini models with reasoning = medium.

Experimental Setup

Five Quality Dimensions Assessed Across Two Models

Readability

LOC per function

Simplicity

Function max
CC

Maintainability

Weighted
maintainability
index

Modularity

Propagation
cost

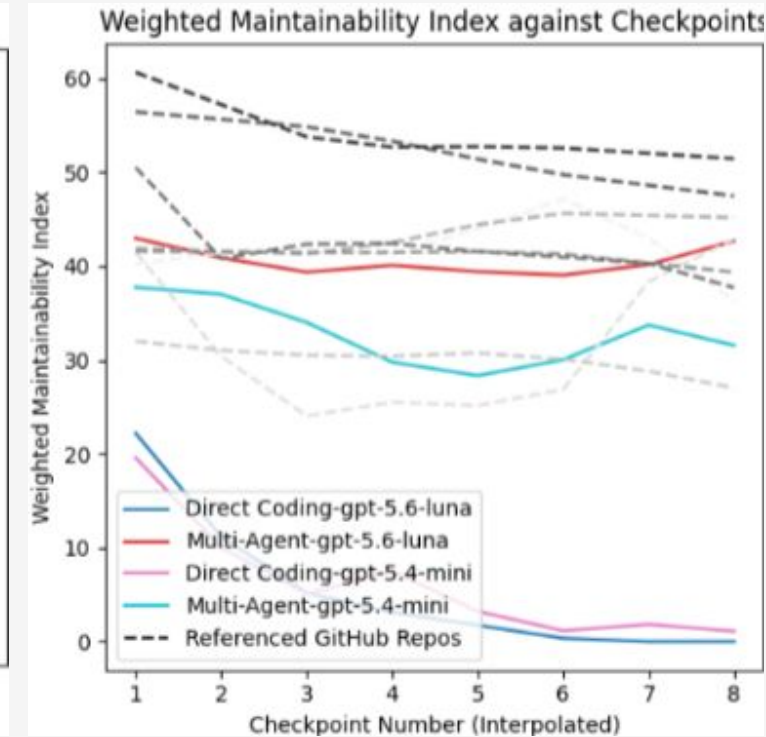
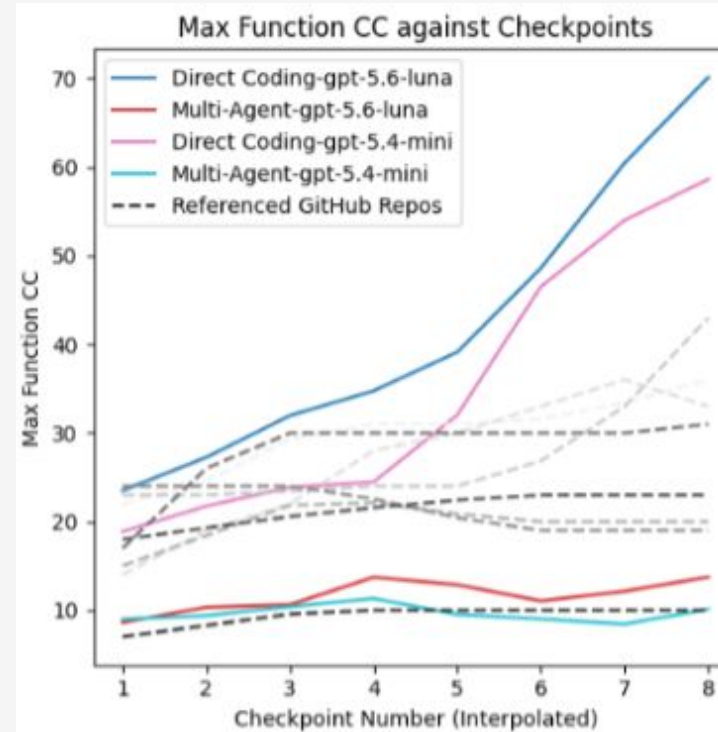
Reusability

Code
duplication

RQ1: Effectiveness in Mitigating Technical Debt

Multi-agent Code is Less Complex and More Maintainable

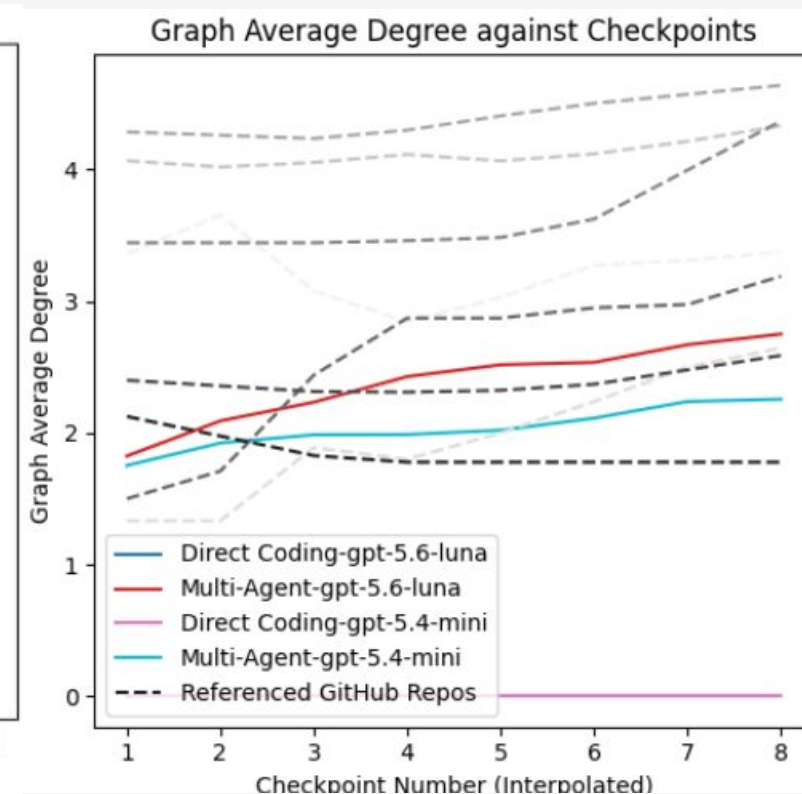
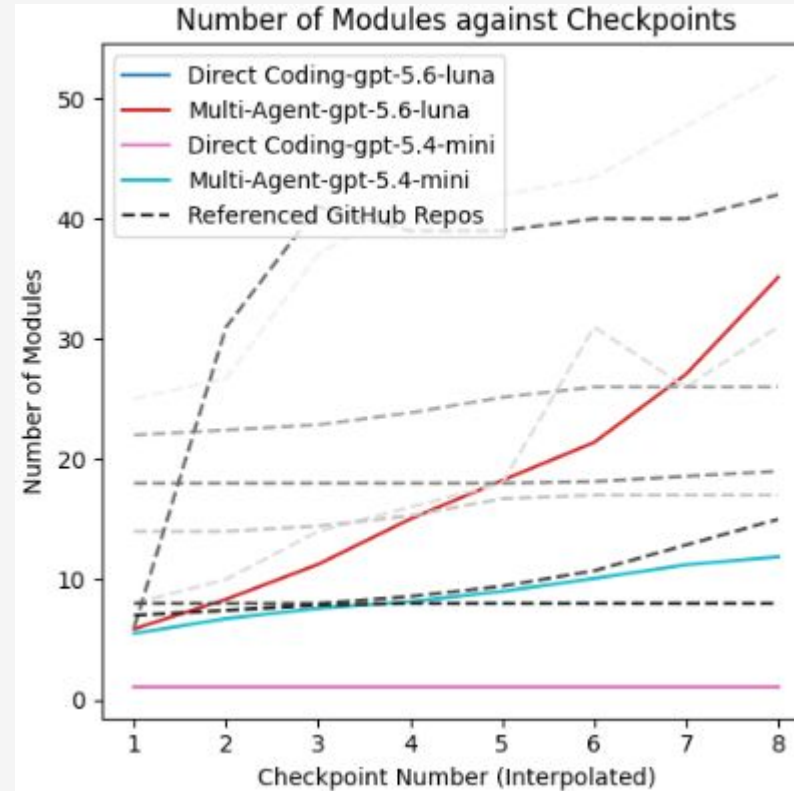
- Reduces the maximum function **CC** by **5×** compared to directly coding.
- **Maintainability index drops by 8%** on average, with the final MI being 37.1, comparable to 40.9 of human-written software.



RQ1: Effectiveness in Mitigating Technical Debt

Multi-agent Code does not Exhibit Evidences of Tight Coupling

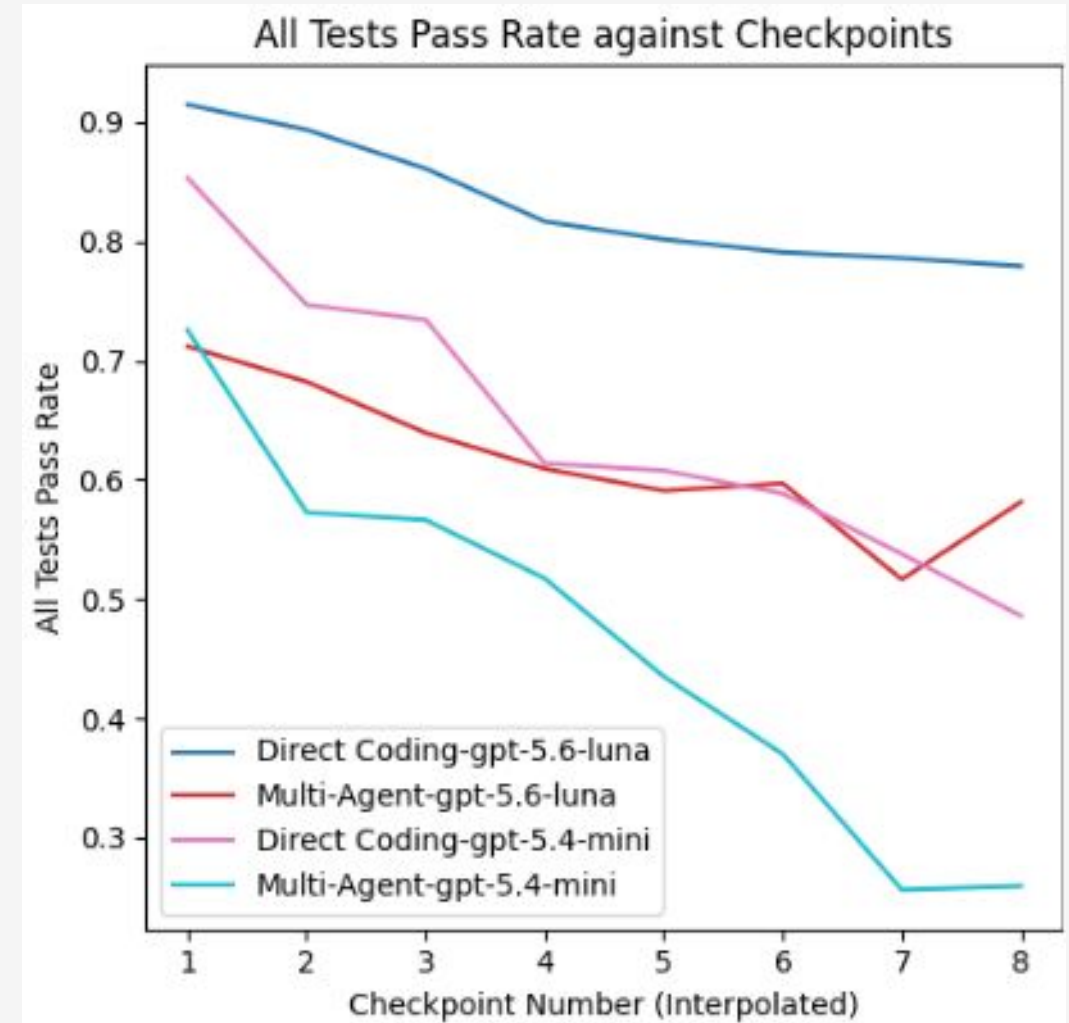
- **Number of modules** increase more quickly for GPT-5.6-luna.
- However, the **graph average degree, propagation cost and impact size**, does not increase proportionally.
- On the other hand, directly coding consistently extends solution in one monolithic module.



RQ2: Better Code Quality Comes with a Correctness Cost

Multi-agent Code Achieve Lower Correctness Across All Problems

- The multi-agent test pass rates at the last checkpoint are **24% and 19% lower** than directly coding for the two models respectively.
- The results are consistent with SlopCodeBench's claim.

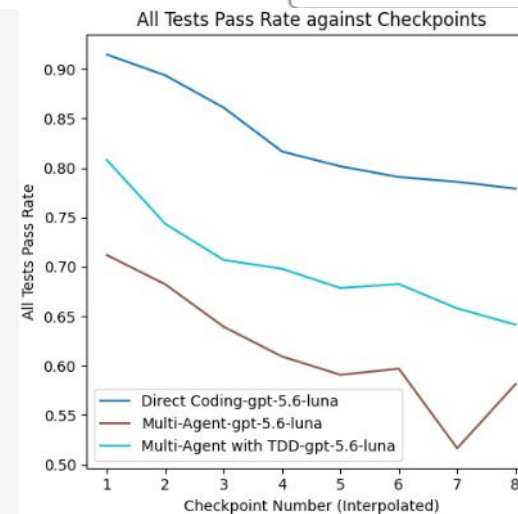
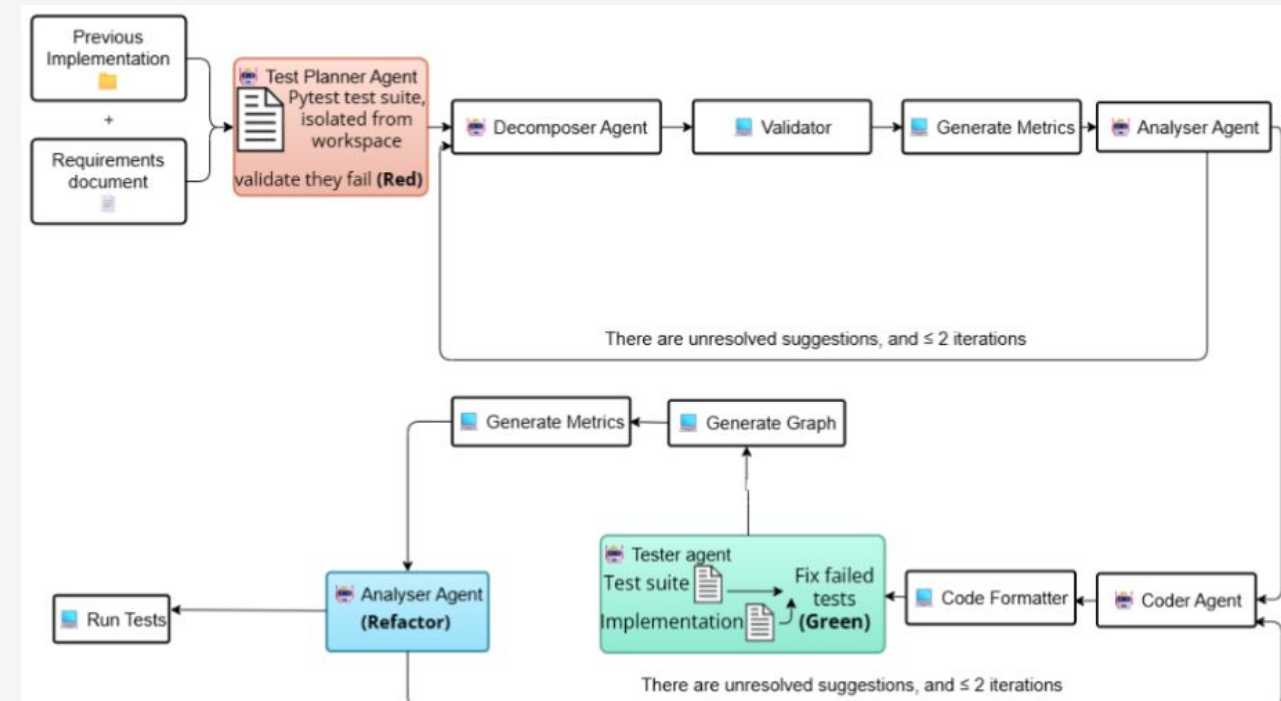


RQ2: Better Code Quality Comes with a Correctness Cost

Test-driven-development Improved the Correctness for Some Models

- Introduces the **test planner** and **tester agent**, that writes black-box tests and fix failed tests respectively.
- This improves the correctness of GPT-5.6-luna by 9%, but the same effect is not observed in the less capable GPT-5.4-mini model.

Mode	All Checkpoints	
	GPT-5.6 Luna	GPT-5.4 Mini
Direct Coding	0.83	0.63
Multi-Agent	0.59	0.44
Multi-Agent+TDD	0.68	0.25



Test pass rates for GPT-5.6-luna

RQ3: Human-in-the-loop Influences Design Decisions

HITL Designs Align More Closely with Expert Intent

- Designed an **iris recognition** problem and provided an expert-designed solution.

Offline (Automatic)

Output csv columns, anomaly csv columns, and dataset input columns are duplicated between different pipeline stages.

HITL

Output, anomaly and dataset input columns are duplicated.

Trade-offs: Remove duplication, but they may intentionally evolve independently.

Question: Should these be extracted into a helper module?

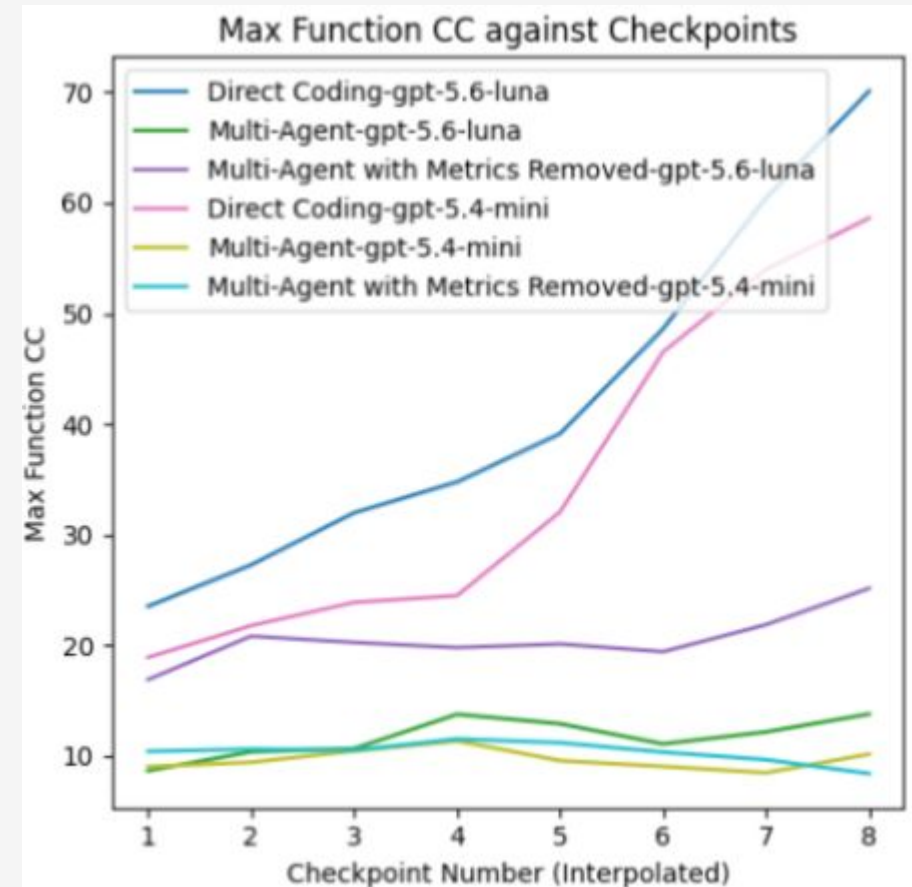
HUMAN:

Yes for output and dataset,
no for anomaly.

Ablation Study: Risk of Goodhart's Law

Quality Gains Cannot be Solely Attributed to Implementation-level Metrics

- Metrics used for optimisation target overlaps with evaluation metrics, such as the maintainability index.
- We remove the **implementation metrics** component.
- Although the final function max CC for GPT-5.6-luna is 82.7% higher, it is still substantially lower than direct coding.



Conclusions and Future Work

Our Contributions

- **Proposed a multi-agent framework** featuring two feedback loops and human involvement.
- **Selected benchmarks and GitHub repositories** for comparison.
- **Evaluated our approach** against direct coding and human-written repositories, showing that it effectively mitigates iterative quality degradation.
- **Demonstrated how the human can guide agent refactoring** by providing context-dependent judgments.

Future Work

- **Use a more diverse set of models** with stronger capabilities.
- **Perform multiple independent runs** to minimise variations caused by LLM's non-deterministic nature.
- **Measure maintainability other than static analysis**, such as using development time or developer perceptions.
- **Survey human participants** for a human alignment score in HITL.

IMPERIAL

Demonstration

A Multi-Agent LLM Framework for
Maintaining Code Quality in Iterative Software Development

10/09/2026

IMPERIAL

Thank You

10/09/2026

References

- ¹ Bogdanov, M. AI Coding Agents: Adoption Trends. JetBrains Research, 2026.
- ² He, H. et al. Speed at the Cost of Quality. MSR '26, 2026.
- ³ Orlanski, G. et al. SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks.
- ⁴ Sharma, T. & Spinellis, D. Do We Need Improved Code Quality Metrics?